OSAWARE — AI & Web Reference

How OSAWARE BASIC talks to Claude, and how a program reaches out to the open web. All of this is client-side: you supply your own Anthropic API key, the browser calls `api.anthropic.com` directly, and the optional backend is not involved.

Build that introduced this set of features: cachebuster `1778619290`. In-app help: `HELP AI`, `HELP WEB` (and `HELP AISYSTEM`, `HELP AIWEB`, … all alias to `HELP AI`).

---

1. Quick start

```
AIKEY                            ' you type this, then paste your sk-ant-… key (masked)
AI "Write one cheerful sentence."  ' streams the reply into the terminal
```

Pull a value into a program variable instead of printing it:

```
10 AI "Name one planet, just the name.", P$
20 PRINT "The AI picked: "; P$
```

Let Claude look something up for you:

```
10 AIWEB ON
20 AI "What is the price of bitcoin right now?", P$
30 PRINT P$
```

Fetch a URL directly (no AI involved):

```
10 WEBGET "wttr.in/Berlin?format=3", W$
20 IF WEBERR$<>"" THEN PRINT "weather down: "; WEBERR$ : END
30 PRINT W$
```

---

2. The AI commands

Command	What it does
`AIKEY`	Prompt for your Anthropic API key with **masked** input. Stored in memory **for this session only — never saved**; page refresh = re-enter it. `AIKEY "sk-ant-…"` works too but echoes the key in plaintext, so prefer the bare form. The key only ever goes in the HTTP header, never into the conversation, so the model can't read it.

Command	What it does
`AI <prompt> [, RESULT$]`	Send `<prompt>` to Claude. With no `RESULT$` it **streams** the reply into the terminal. With `RESULT$` (a string variable) it runs silently and stores the reply there. Conversation history is kept across calls (auto-trimmed at ~40 messages); `AICLEAR` resets it.
`AINUM <prompt>, VAR`	Like `AI` but `VAR` is a **numeric** variable. The model is told to reply with only a number; if the reply isn't numeric you get `0`. Best paired with `AITEMP 0`.
`AISYSTEM "text"` / `AISYSTEM VAR$` / `AISYSTEM @"file"` / `AISYSTEM ""` / `AISYSTEM`	Set the **system prompt** — Claude's persona, your standing instructions, raw context/data it should always know. Your text **replaces** the built-in default (a short "you're an assistant in a BASIC terminal" line); a "reply in plain text, no markdown" instruction is always appended on top. `AISYSTEM @"file"` loads the prompt from a **saved VFS file/program** — its line text joined with newlines, line numbers stripped (`SAVE "OSDATA"` first, then `AISYSTEM @"OSDATA"`). `AISYSTEM ""` resets to the default; `AISYSTEM` with no argument prints the current one.
`AIMODEL name` / `AIMODEL`	Pick the model. Aliases: `FAST` / `HAIKU` (the default), `SMART` / `SONNET`, `BEST` / `OPUS`, `DEFAULT`. Anything else is used as a literal Anthropic model id. `AIMODEL` with no argument shows the current one. Default model: `claude-haiku-4-5-20251001`.
`AITEMP n` / `AITEMP`	Sampling temperature, `0`–`1`. `0` = deterministic — what you want for pulling data so the same prompt gives the same answer. An out-of-range value resets to the API default. `AITEMP` with no argument shows it.
`AITOKENS n` / `AITOKENS`	Max output tokens, `1`–`8192`. Default `1024` (`4096` automatically while `AIWEB` is on, since web answers run long). `0` or a bad value resets to `1024`. `AITOKENS` with no argument shows it.
`AIWEB ON` / `AIWEB OFF` / `AIWEB`	When **ON**, `AI` and `AINUM` requests carry Anthropic's server-side **web search** tool (`web_search_20250305`, up to 5 searches per answer). Claude decides whether and what to search, Anthropic runs the search, and you get a synthesised answer in one call. While on: the timeout stretches to 60 s, `AITOKENS` defaults to `4096`, and the system prompt gets a nudge to look things up rather than guess. **OFF by default** — web search adds latency and a small per-search cost. `AIWEB` with no argument shows the state.
`AICLEAR` / `AICLEAR ALL`	`AICLEAR` wipes the conversation history. `AICLEAR ALL` also resets `AISYSTEM` / `AIMODEL` / `AITEMP` / `AITOKENS` / `AIWEB` back to defaults.

`AIERR$` — the AI error flag

After **every** `AI` or `AINUM` call, the string variable `AIERR$` is set:

- `""` on success
- the error message on failure (no key, network down, rate limit, model rejected a tool, …)

On failure the result variable is also given a defined value — `""` for `AI`, `0` for `AINUM` — so a program always has something safe to test:

```
10 AI "What is the capital of France?", C$
20 IF AIERR$ <> "" THEN PRINT "AI is down: "; AIERR$ : END
30 PRINT "Capital: "; C$
```

`AISYSTEM` vs `AIWEB`

- `AISYSTEM` shapes **who Claude is and what it knows** going in (persona, format rules, your data).
- `AIWEB ON` gives Claude the **ability to look things up** while answering.

They're independent and they combine well — a custom persona that also has live web access.

What goes in a request

Each `AI` / `AINUM` call sends, to `https://api.anthropic.com/v1/messages`:

- `model` — `AIMODEL`'s value, or `claude-haiku-4-5-20251001`
- `max_tokens` — `AITOKENS`'s value, or `1024` (`4096` if `AIWEB` is on)
- `system` — `AISYSTEM`'s text (or the built-in default) + a "plain text only, no markdown" line + (`AINUM` only) a "reply with only a number" line + (`AIWEB` on) a "use the web search tool" line
- `temperature` — only if `AITEMP` was set
- `messages` — the running conversation
- `tools` — `[{ web_search }]` only if `AIWEB` is on
- `stream: true` — the reply streams in token by token

---

3. The web command

Command	What it does
`WEBGET url$, RESULT$`	HTTP **GET**. `url$` may be a quoted string or a string variable; `https://` is assumed if you leave off the scheme. The response body text goes into `RESULT$` (capped at 1 MB, with a `…[truncated at 1 MB]` marker if longer). 20-second timeout. Execution pauses until the response arrives (same as `AI` / `LOAD` / `WS.OPEN`).

After a `WEBGET`:

- `WEBSTATUS` — numeric — the HTTP status code (`200`, `404`, …), or `0` on a network / CORS error or timeout
- `WEBERR$` — string — `""` on success, or the error message (e.g. `Failed to fetch`, `HTTP 404`, `timeout (20s)`). On a failure the result variable is also set to `""`.

```
10 WEBGET "api.github.com/repos/jahshaka/OSaware", J$
20 IF WEBERR$<>"" THEN PRINT "fetch failed: "; WEBERR$ : END
30 P = INSTR(J$, "stargazers_count")
40 PRINT MID$(J$, P, 40)
```

The CORS limit (read this)

The browser only lets `WEBGET` reach sites that send the `Access-Control-Allow-Origin` header. Most public JSON APIs do — `api.github.com`, `wttr.in`, `api.coingecko.com`, the Wikipedia API, `jsonplaceholder.typicode.com`, and so on. Many ordinary web pages and stricter APIs do **not**, and those fail

with `Failed to fetch`. The only way past this is a server-side proxy (a future `/api/fetch?url=…` on the OSAWARE backend); `WEBGET` itself is GET-only for now.

`WEBGET` vs `AIWEB`

- `WEBGET url$, R$` — your program fetches a **URL you name**; you get the **raw bytes** in `R$`. Direct and free, but CORS-limited and you parse it yourself.
- `AIWEB ON` + `AI "…"` — Claude figures out **what to look up**, does it (Anthropic does the fetching, so no CORS limit), and hands you a **synthesised answer**. Costs an API call plus a search fee, and you get prose, not raw JSON.

Use `WEBGET` when you know the exact endpoint and want the data; use `AIWEB` when you want Claude to do the legwork.

---

4. Things to know

- **One command per line.** `AI`, `AINUM`, `AISYSTEM`, `AIMODEL`, `AITEMP`, `AITOKENS`, `AIWEB`, `AIKEY`, `AICLEAR`, and `WEBGET` each consume the rest of their line. `AIWEB ON : AITEMP 0` does **not** work — `AIWEB` swallows `: AITEMP 0`. Put each on its own line; do the error check on the next line.
- **Prompts aren't expressions.** The string argument to `AI`, `AISYSTEM`, `WEBGET` is either a quoted literal `"…"` or a single variable name. To build a dynamic prompt, concatenate into a variable first: `P$ = "Tell me about " + TOPIC$ : AI P$, R$`.
- **Execution freezes during the call.** While an `AI` / `AINUM` / `WEBGET` (or an async `AISYSTEM @"file"`) is in flight, the program is paused — same as `LOAD`, image loads, and `WS.OPEN`. It resumes on the next line when the response (or error) arrives.
- **The key isn't persisted.** Refresh the page and you re-enter it with `AIKEY`.
- **Web search and the model.** Web search is generally available on the current Claude models, so the default (Haiku 4.5) should handle it. If a model rejects it you'll see `AI ERROR: … does not support tool web_search` — switch with `AIMODEL SMART`.
- **`AINUM` can still misfire.** If the model returns "$97,234 as of 14:30" instead of `97234`, `AINUM` gives `0`. Tighten the prompt ("just the number, no commas, no symbols"), set `AITEMP 0`, or fall back to `AI …, P$` and display the string.
- **Long sessions.** `AI` / `AINUM` keep a conversation; it auto-trims at ~40 messages so it can't blow the token limit. `AICLEAR` between unrelated questions keeps each one clean.
- **AISYSTEM config persists for the session.** Set `AISYSTEM` / `AIMODEL` / `AITEMP` / `AIWEB` once and they stay until you change them, `AICLEAR ALL`, or refresh. A program that wants a known starting point should set them at the top.

---

5. Worked example — LIVE BRIEFING

A retro dashboard that pulls today's data through Claude's web search. Type `AIKEY` first, then `RUN`. `SAVE "BRIEF"` to keep it.

```
10  REM ============================================================
20  REM   LIVE BRIEFING  --  a retro dashboard wired into the live web,
30  REM   via Claude's web-search tool.
40  REM   Before RUN:  type  AIKEY  and paste your Anthropic key.
50  REM ============================================================
60  REM --- give Claude a role, and let it search the web ---
70  AISYSTEM "You are BRIEFING, a terse live-data feed bolted onto an old home computer. ALWAYS use the web search tool to get current, real data. Reply with ONLY the value or short phrase asked for -- no preamble, no explanation, no citations, no markdown, no quotation marks. If you genuinely cannot find it, reply exactly: UNKNOWN"
80  AIWEB ON
90  AITEMP 0
100 CITY$ = "London"
110 REM --- (re)draw the dashboard ---
120 CLS : COLOUR 14
130 PRINT "  +-----------------------------------------+"
140 PRINT "  |           L I V E   B R I E F I N G     |"
150 PRINT "  +-----------------------------------------+"
160 COLOUR 7 : PRINT : PRINT "   ...asking around, one moment..." : PRINT
170 REM --- weather ---
180 P$ = "Current weather in " + CITY$ + " as one short phrase, e.g. 8C light rain"
190 AI P$, A$
200 IF AIERR$<>"" THEN A$ = "(unavailable: " + AIERR$ + ")"
210 PRINT "   Weather, " + CITY$ + " :  " + A$
220 REM --- bitcoin ---
230 AINUM "Bitcoin price in US dollars right now -- just the number, no commas, no symbols", N
240 IF AIERR$<>"" THEN PRINT "   Bitcoin (USD)      :  (unavailable)"
250 IF AIERR$="" THEN PRINT "   Bitcoin (USD)      :  $"; INT(N)
260 REM --- headline ---
270 AI "The single biggest world news headline right now, max 55 chars, no quotes", A$
280 IF AIERR$<>"" THEN A$ = "(unavailable)"
290 PRINT "   Top story          :  " + A$
300 REM --- on this day ---
310 AI "One notable historical event that happened on today's calendar date, under 55 chars", A$
320 IF AIERR$<>"" THEN A$ = "(unavailable)"
330 PRINT "   On this day        :  " + A$
340 PRINT : COLOUR 11
350 PRINT "   [R] refresh     [C] change city     [Q] quit"
360 COLOUR 7
370 K = GETKEY()
380 IF K=81 OR K=113 THEN CLS : COLOUR 7 : PRINT "Stay curious." : END
390 IF K=67 OR K=99 THEN GOTO 420
400 GOTO 110
410 REM --- change city ---
```

```
420 PRINT
430 INPUT "   New city: "; C$
440 IF C$<>"" THEN CITY$ = C$
450 GOTO 110
```

A second pattern — a character with a custom system prompt that also web-checks its facts:

```
10  AISYSTEM "You are CAPTAIN, a salty old sea captain. Use the web search tool for any real facts.
Answer in character -- gruff, brief, a bit of nautical flavour -- but the facts must be accurate.
Plain text, no markdown."
20  AIWEB ON
30  PRINT : INPUT "Ask the Captain (blank to quit): "; Q$
40  IF Q$="" THEN PRINT "Fair winds." : END
50  AI Q$, R$
60  IF AIERR$<>"" THEN PRINT "The Captain's radio is down: "; AIERR$ : END
70  PRINT : PRINT R$
80  GOTO 30
```

Loading a big context blob from a file:

```
' Type your facts as a numbered "program", then SAVE it:
10 We are OSAWARE - a browser-based OS with a BASIC interpreter.
20 Founder: Karsten Becker.  Live at osaware.com.
30 Frontend is plain JS; an optional Node backend adds accounts + cloud storage.
SAVE "OSDATA"

' Then in any program:
10 AISYSTEM @"OSDATA"
20 AITEMP 0
30 AI "Who runs OSAWARE?", R$
40 PRINT R$
```

---

6. Where it lives (developer notes)

- `core/kernel.js` — `cmdAIKEY`, `cmdAICLEAR`, `cmdAISYSTEM` + `_aiSystemFromFile`, `cmdAIMODEL`, `cmdAITEMP`, `cmdAITOKENS`, `cmdAIWEB`, `cmdAI` / `cmdAINUM` → `_aiDispatch`, `_callAnthropicAPI`, `cmdWEBGET`; the `['AI…']` / `['WEBGET']` command-table entries; the `ai_key / ai_messages / ai_system / ai_model / ai_temp / ai_tokens / ai_web / _aiDefaultModel` state in the constructor.
- `core/shell.js` — the `HELP AI` and `HELP WEB` topics; the `AI:` / `WEB:` lines in the main `HELP`.
- `core/drivers/terminal.js` — the `__AIKEY__` masked-input handler.
- The async-pause pattern (`want_ai` flag) is shared with `LOAD`, image loading, `WS.OPEN`, and the auth ops.

- After editing `core/*.js`, run `./bump.sh` to bump the cachebuster so the browser picks up the change.